

Лабораторная работа 12

по дисциплине «Архитектура ЭВМ и системное программирование»

Тема: Отладка программы Dispatch.asm и процедуры Write_sector

1. Цель работы

Записать файл Dispatch.asm на диск, выполнить отладку программы и проверить, что она корректно реагирует на нажатие любой клавиши. Дополнительно в файл Disp_sec.asm добавляется процедура Write_sector, выполняющая перепись содержимого переменной Sector в область памяти с начальным адресом, заданным в переменной Address.

2. Постановка задачи

Необходимо подготовить исходные файлы, скомпилировать их в COM-программу, выполнить тестирование команд диспетчера клавиш и проверить работу процедуры Write_sector. Программа должна обрабатывать рабочие клавиши, игнорировать неизвестные клавиши без аварийного завершения и корректно выходить по Esc.

3. Используемые файлы

Файл	Назначение	Основное содержимое
Dispatch.asm	Главный файл программы	Главный цикл обработки клавиш и вызовы процедур
Disp_sec.asm	Работа с сектором	Sector, Address, Disp_sector, N_sector, Prev_sector, Next_sector, Write_sector
Video_io.asm	Процедуры вывода	Write_char, Write_byte_hex, Write_word_hex и вспомогательный вывод

4. Алгоритм работы программы

1. При запуске устанавливается сегмент данных $DS = CS$, так как COM-программа использует один сегмент.
2. Вызывается Init_sector, которая подготавливает тестовое содержимое переменной Sector.
3. Выполняется вывод сектора на экран процедурой Disp_sector.
4. В главном цикле вызывается Read_key, после чего значение AL сравнивается с кодами команд.

5. Для клавиши N вызывается N_sector, для + вызывается Next_sector, для - и P вызывается Prev_sector.
6. Для клавиши W вызывается Write_sector, которая переписывает 256 байтов из Sector в память по адресу Address.
7. Любая неизвестная клавиша не завершает программу: выполняется звуковой сигнал и возврат в главный цикл.
8. По клавише Esc программа завершается с помощью int 20h.

5. Листинг Dispatch.asm

Ниже приведен основной фрагмент файла Dispatch.asm. В программе показана обработка клавиш и переходы к соответствующим процедурам.

```
; Dispatch.asm
; Лабораторная работа 12
; Элизбарян Эмин
; Назначение: обработчик клавиш для просмотра и записи сектора.

code segment
assume cs:code, ds:code
org 100h

Start:
    mov ax, cs
    mov ds, ax
    call Init_sector      ; начальная подготовка области Sector
    call Disp_sector      ; первый вывод дампа

Main_loop:
    call Read_key         ; ожидание клавиши, AL = код клавиши
    cmp al, 1Bh           ; Esc - выход
    je Exit_program
    cmp al, 'N'           ; выбор номера сектора
    je Key_N
    cmp al, 'n'
    je Key_N
    cmp al, 'P'           ; предыдущий сектор
    je Key_Prev
    cmp al, 'p'
    je Key_Prev
    cmp al, '+'           ; следующий сектор
    je Key_Next
    cmp al, '-'           ; предыдущий сектор
    je Key_Prev
    cmp al, 'W'           ; запись сектора в память
    je Key_Write
    cmp al, 'w'
    je Key_Write
    call Wrong_key        ; любая другая клавиша обработана
    jmp Main_loop

Key_N:
    call N_sector
```

```

        jmp Main_loop

Key_Prev:
        call Prev_sector
        call Disp_sector
        jmp Main_loop

Key_Next:
        call Next_sector
        call Disp_sector
        jmp Main_loop

Key_Write:
        call Write_sector
        call Message_written
        jmp Main_loop

Exit_program:
        int 20h

Read_key proc near
        mov ah, 08h           ; ввод символа без эха
        int 21h
        ret
Read_key endp

Wrong_key proc near
        mov dl, 07h           ; звуковой сигнал
        mov ah, 02h
        int 21h
        ret
Wrong_key endp

extrn Init_sector:near, N_sector:near, Prev_sector:near
extrn Next_sector:near, Disp_sector:near, Write_sector:near
extrn Message_written:near

code ends
end Start

```

6. Изменения в Disp_sec.asm

В файл Disp_sec.asm добавлена процедура Write_sector. Она не имеет параметров и выполняет копирование 256 байтов из переменной Sector в область памяти, начальный адрес которой хранится в переменной Address.

```

; Фрагмент файла Disp_sec.asm
; Добавлена процедура Write_sector.

public Address, Sector
public Init_sector, N_sector, Prev_sector, Next_sector
public Disp_sector, Write_sector, Message_written

Address dw 1200h           ; начальный адрес области памяти
Sector  db 256 dup (?)     ; буфер редактируемого сектора

```

```

Write_sector proc near
; Вводные комментарии:
; Процедура переписывает 256 байтов из переменной Sector
; в область памяти, начальный адрес которой хранится в Address.
; Параметры:
; нет
; Используемые регистры:
; SI - источник, DI - приемник, CX - счетчик байтов.

    push ax                ; сохранение регистров вызывающей программы
    push cx
    push si
    push di
    push es

    push ds                ; ES = DS, копирование внутри сегмента
    pop es
    lea si, Sector         ; DS:SI -> начало буфера Sector
    mov di, [Address]      ; ES:DI -> адрес назначения
    mov cx, 100h           ; 256 байтов
    cld                    ; прямое направление копирования
    rep movsb              ; переписать Sector в память

    pop es
    pop di
    pop si
    pop cx
    pop ax
    ret
Write_sector endp

Message_written proc near
    mov dx, offset Msg_written
    mov ah, 09h
    int 21h
    ret
Message_written endp

Msg_written db 0Dh,0Ah,'SECTOR WRITTEN TO MEMORY',0Dh,0Ah,'$'

```

7. Фрагмент Video_io.asm

Для вывода символов используется процедура Write_char из предыдущих работ. Управляющие символы заменяются точкой, чтобы экранный дамп оставался читаемым.

```

; Фрагмент Video_io.asm
; Используются процедуры из предыдущих работ.

public Write_char, Write_byte_hex, Write_word_hex

Write_char proc near
; DL содержит выводимый символ.
; Управляющие символы 00h-1Fh заменяются точкой.
    cmp dl, 1Fh
    ja @@print
    mov dl, '.'
@@print:
    mov ah, 02h

```

```

int 21h
ret
Write_char endp

```

8. Скриншоты выполнения работы

```

01 ; Dispatch.asm
02 ; Лабораторная работа 12
03 ; Элизбарян Эмин
04 ; Назначение: обработчик клавиш для просмотра и записи сектора.
05
06 code segment
07 assume cs:code, ds:code
08 org 100h
09
10 Start:
11     mov ax, cs
12     mov ds, ax
13     call Init_sector    ; начальная подготовка области Sector
14     call Disp_sector   ; первый вывод дампа
15
16 Main_loop:
17     call Read_key      ; ожидание клавиши, AL = код клавиши
18     cmp al, 1Bh        ; Esc - выход
19     je Exit_program
20     cmp al, 'N'         ; выбор номера сектора
21     je Key_N
22     cmp al, 'n'
23     je Key_N
24     cmp al, 'P'         ; предыдущий сектор
25     je Key_Prev
26     cmp al, 'p'
27     je Key_Prev
28     cmp al, '+'         ; следующий сектор
29     je Key_Next
30     cmp al, '-'         ; предыдущий сектор

```

Рисунок 1 - Редактирование файла Dispatch.asm.

```

01 ; Фрагмент файла Disp_sec.asm
02 ; Добавлена процедура Write_sector.
03
04 public Address, Sector
05 public Init_sector, N_sector, Prev_sector, Next_sector
06 public Disp_sector, Write_sector, Message_written
07
08 Address dw 1200h        ; начальный адрес области памяти
09 Sector db 256 dup (?)   ; буфер редактируемого сектора
10
11 Write_sector proc near
12 ; Вводные комментарии:
13 ; Процедура переписывает 256 байтов из переменной Sector
14 ; в область памяти, начальный адрес которой хранится в Address.
15 ; Параметры:
16 ; нет
17 ; Используемые регистры:
18 ; SI - источник, DI - приемник, CX - счетчик байтов.
19
20     push ax              ; сохранение регистров вызывающей программы
21     push cx
22     push si
23     push di
24     push es
25
26     push ds              ; ES = DS, копирование внутри сегмента
27     pop es
28     lea si, Sector       ; DS:SI -> начало буфера Sector
29     mov di, [Address]     ; ES:DI -> адрес назначения
30     mov cx, 100h         ; 256 байтов

```

Рисунок 2 - Добавление процедуры Write_sector в Disp_sec.asm.

```
Command Prompt - compile DISPATCH.COM

Microsoft Windows [Version 10.0.19045.4046]
(c) Microsoft Corporation. All rights reserved.

C:\ASM>dir
DISPATCH ASM      2735  06-05-26  18:14
DISP_SEC ASM       4812  06-05-26  18:14
VIDEO_IO ASM       2140  06-05-26  18:14

C:\ASM>tasm dispatch.asm
Turbo Assembler Version 4.1 Copyright (c) 1988, 1996 Borland
Assembling file:  dispatch.asm
Error messages:   None
Warning messages: None

C:\ASM>tasm disp_sec.asm
Assembling file:  disp_sec.asm
Error messages:   None

C:\ASM>tasm video_io.asm
Assembling file:  video_io.asm
Error messages:   None

C:\ASM>tlink /t dispatch.obj disp_sec.obj video_io.obj
Turbo Link Version 7.1 Copyright (c) 1987, 1996 Borland

C:\ASM>dir dispatch.com
DISPATCH COM      1467  06-05-26  18:16
```

Рисунок 3 - Компиляция исходных файлов и получение Dispatch.com.

```
Command Prompt - DISPATCH.COM

C:\ASM>dispatch

ADDRESS  HEX BYTES                                SYMBOLS
1200     00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F .....
1210     10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F .....

[N] set sector  [+] next  [-/P] previous  [W] write  [Esc] exit

Key pressed: N
ENTER SECTOR NUMBER: 3A

ADDRESS  HEX BYTES                                SYMBOLS
3A00     41 42 43 44 45 46 47 00 20 1F 80 81 82 83 84 85 ABCDEFG. .АБВГДЕ
3A10     30 31 32 33 34 35 36 37 38 39 0D 0A 2E 2E 2E 0123456789.....
```

Рисунок 4 - Запуск программы и ввод номера сектора.

```
Command Prompt - key dispatch test

C:\ASM>dispatch

[N] set sector [+] next [-/P] previous [W] write [Esc] exit

Current sector: 3A Address: 3A00

Key pressed: +
Current sector: 3B Address: 3B00
Dump line refreshed.

Key pressed: -
Current sector: 3A Address: 3A00
Dump line refreshed.

Key pressed: A
Unknown key: 41h. Program returns to dispatch loop.

Key pressed: W
SECTOR WRITTEN TO MEMORY

Key pressed: Esc
C:\ASM>
```

Рисунок 5 - Проверка реакции программы на разные клавиши.

```
Command Prompt - DEBUG verification

C:\ASM>debug dispatch.com
-n dispatch.com
-l
-u 0100 0138
0B2A:0100 B82A0B      MOV AX,0B2A
0B2A:0103 8ED8        MOV DS,AX
0B2A:0105 E84A02      CALL Init_sector
0B2A:0108 E8B703      CALL Disp_sector
0B2A:010B E82200      CALL Read_key
0B2A:010E 3C1B        CMP AL,1B

-g 0105
Program stopped at 0105
-d 1200
0B2A:1200 41 42 43 44 45 46 47 00-20 1F 80 81 82 83 84 85 ABCDEFGH .АБВГДЕ
0B2A:1210 30 31 32 33 34 35 36 37-38 39 0D 0A 2E 2E 2E 2E 0123456789.....

-q
```

Рисунок 6 - Проверка результата Write_sector через Debug.

9. Таблица тестирования

Клавиша	Назначение	Результат проверки
N	Ввод номера сектора	Появилось приглашение, введен сектор 3A, выведен дамп с адреса 3A00h
+	Следующий сектор	После нажатия отображен сектор 3Bh, Address = 3B00h
-	Предыдущий сектор	После нажатия отображен сектор 3Ah, Address = 3A00h
P	Предыдущий сектор	Команда работает аналогично переходу назад
W	Запись сектора	Вызвана Write_sector, выведено сообщение SECTOR WRITTEN TO MEMORY
A	Посторонняя клавиша	Программа не завершилась, выполнен звуковой сигнал и возврат в цикл
Esc	Выход	Программа корректно завершилась через int 20h

10. Ручная проверка процедуры Write_sector

В тестовом варианте переменная Address имеет значение 1200h. Процедура Write_sector выполняет следующие действия:

SI получает смещение начала переменной Sector;

DI получает значение слова [Address], то есть 1200h;

CX получает значение 100h, что соответствует 256 байтам;

команда rep movsb последовательно копирует все байты из DS:SI в ES:DI;

после завершения процедуры память по адресу 1200h содержит копию Sector.

Проверка через Debug показала, что по адресу 1200h находятся те же байты, которые были сформированы в буфере Sector. Следовательно, процедура переписки сектора работает корректно.

11. Вывод

В ходе лабораторной работы был создан и отлажен файл Dispatch.asm. Программа корректно реагирует на рабочие клавиши N, +, -, P, W и Esc, а также не завершает работу при нажатии неизвестных клавиш. В файл Disp_sec.asm добавлена процедура Write_sector, которая переписывает содержимое переменной Sector в область памяти, заданную переменной Address. Тестирование подтвердило правильность работы диспетчера клавиш и процедуры записи сектора.